The Iby and Aladar Fleischman Faculty of Engineering
Tel Aviv University

Velocity Control

HexaMotor



Experiment objectives:

- Introduction to DC motor dynamic equations.
- Coulomb Friction identification and handling
- System identification using open loop step response, model validation.
- System identification using frequency response, model validation.
- Velocity control implementation. Proportional, Proportional - Integral control loops.
- Controller performance under load.

1. Introduction

DC motors are electromechanical devices that convert electrical energy into mechanical energy, using principles of electromagnetism. They are known for their simple construction, high reliability, and ease of control. These features make them suitable for various applications, from small electronic devices to large industrial machinery.

1.1. DC motor equations

The electric equivalent circuit of the armature and the free-body diagram of the rotor are shown in Figure 1.

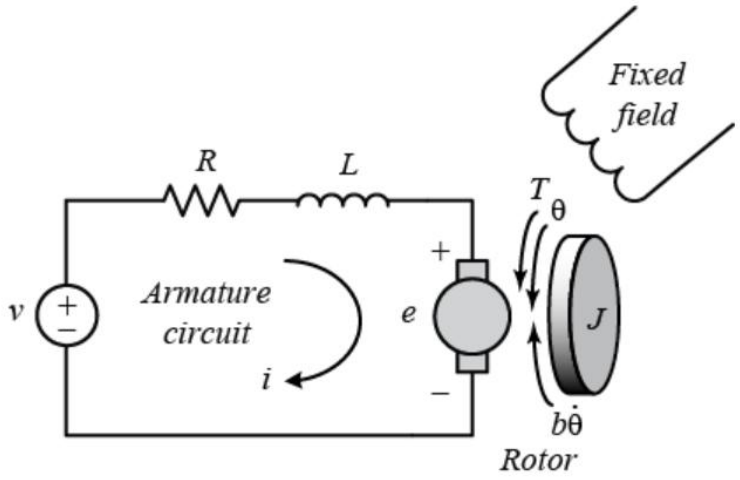


Figure 1: Electric equivalent circuit of a dc motor

V - Applied Voltage: The voltage applied across the motor's terminals. It is the input to the system and determines the driving force behind the motor.

i - Armature Current: The current flowing through the armature winding, responsible for creating the magnetic field that drives the motor.

R - Armature Resistance: The resistance of the armature winding, which opposes the flow of current.

L - Armature Inductance: The inductance of the armature winding, which determines how quickly the current can change in the coil.

$\boldsymbol{V}_{emf}$ - Back Electromotive Force (EMF): Voltage generated by the motor's rotation, opposing the applied voltage.

$\boldsymbol{K}_T$ - Torque Constant: A factor that relates the motor's torque output to its current. It depends on the motor's construction.

$\boldsymbol{K_b}$ - Back EMF Constant: A factor that relates the motor's back EMF to the angular velocity. It often equals the torque constant in many DC motors.

$\boldsymbol{T}$ - Torque: The twisting force exerted by the motor on its shaft.

$\boldsymbol{T_{friction}}$ - Friction Torque: The torque opposing the motor's rotation due to various frictional effects.

$\boldsymbol{J}$ - Moment of Inertia: A measure of the motor's resistance to changes in its rotational speed. It depends on the distribution of mass in the rotating parts.

$\boldsymbol{\omega}$ - Angular Velocity: The rate of rotation of the motor's shaft, often the controlled output in velocity control systems.

$\boldsymbol{B}$ - Damping Coefficient: A parameter representing the viscous friction in the system. It characterizes how the frictional forces are proportional to the angular velocity.

$\boldsymbol{T_{coulomb}}$ - Coulomb friction.

$\boldsymbol{s}$ - Complex Frequency (Laplace Variable): In the context of the transfer function, s is a complex variable used in the Laplace transform.

Voltage Equation:

1.1 $$V = Ri + L\frac{di}{dt} + K_b\omega$$

where V is the applied voltage, R is the armature resistance, $\boldsymbol{i}$ is the armature current, L is the armature inductance, K_b is the back EMF constant, and ω is the angular velocity.

Torque Equation:

1.2 $$T = K_T i - T_{friction} - J\frac{d\omega}{dt}$$

where T is the torque, K_T is the torque constant, $T_{friction}$ is the friction torque, J is the moment of inertia, and $\frac{d\omega}{dt}$ is the angular acceleration.

Friction Model:

1.3 $$T_{friction} = B\omega + T_{coulomb}$$

By substituting the expressions for torque and friction into the torque equation and solving for the differential equation relating the applied voltage to the angular velocity, we obtain:

1.4 $$LJ\frac{d^2\omega}{dt^2} + (RJ + BL)\frac{d\omega}{dt} + (BR + K_T K_b)\omega = K_T V$$

Now, the transfer function relating the applied voltage *V(s)* to the angular velocity *Ω(s)* in the Laplace domain (assuming zero initial conditions) is:

1.5 $$G(s) = \frac{K_T}{LJs^2 + (Rj + BL)s + (BR + K_T K_b)}$$

1.2. First order approximation:

Armature Inductance (L): The inductive term $L\frac{di}{dt}$ in the voltage equation is often neglected. This is under the assumption that the armature inductance is small compared to the resistance, meaning that the electrical dynamics are much faster than the mechanical dynamics.

First-Order Transfer Function:

1.6 $$G(s) = \frac{K_T}{(RJ)s+(BR+K_TK_b)}$$

This first-order model offers a simpler representation of the DC motor's dynamics and is useful for preliminary control design or when the details of the electrical subsystem are not critical. It captures the main characteristics of the motor's velocity response but omits some of the subtler dynamics present in the full second-order model.

For additional reading consider the following links:

DC motor velocity model:

https://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed&section=SystemModeling

System analysis:

https://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed&section=SystemAnalysis

1.3. Coulomb friction

Coulomb friction is distinctive from viscous damping, as it remains constant regardless of the speed of the motor. It contributes to non-linearity in the system and can lead to challenges in maintaining precise control, especially at low speeds.

Measuring Coulomb friction in a DC motor is essential for designing effective control strategies. One common method to identify this friction involves applying a small, controlled voltage to the motor and observing the threshold voltage at which the motor begins to rotate. By relating this threshold voltage to the known torque-to-current relationship, the Coulomb friction torque can be quantified.

1.4. Velocity measurement with an encoder

The HexaMotor measures the motor angle with a quadrature encoder: two channels (A and B) produce pulses while the motor turns, and every rising and falling edge of both channels is one tick (4 ticks per encoder line). The angular velocity is not measured directly; it is calculated from the ticks. Two methods are used in this lab, and both are shown on the scope:

- **Tick-count velocity:** the ticks are counted every sample period Ts and the velocity is the change of the angle between samples, $\omega = 2\pi \cdot \Delta\text{ticks} / (\text{CPR} \cdot \text{Ts})$. With only a few ticks per sample the result is coarse (one tick more or less changes the velocity by a large step), so it is smoothed with a low pass filter (Filtered Derivative block, $\tau = 0.05$ s). In the models this is the "Tick-count velocity" signal (scope column 2).
- **Tick-timing velocity:** the time between consecutive ticks is measured with microsecond resolution in an interrupt, and the velocity is the inverse of the average tick interval, $\omega = 2\pi / (\text{CPR} \cdot \Delta t)$. The HexaMotor Encoder block averages the tick intervals of the last 10 ms (up to 48 ticks, one motor revolution), rejects single deviating ticks and applies a light 5 ms smoothing. In the models this is the "Tick-timing velocity" signal (scope column 3).

The better method depends on the encoder resolution. A high resolution encoder (thousands of counts per revolution) produces many ticks in every sample period, so counting the ticks per sample is already precise and needs only light filtering; timing every tick would also load the processor with a very high interrupt rate. A low resolution encoder produces only a few ticks per sample: counting gives a coarse velocity that needs strong filtering, while timing the ticks still gives a precise value at every tick.

Filtering always adds a time delay. A low pass filter with time constant τf delays slow signals by about τf and adds a phase lag of $\text{atan}(\omega \cdot \tau f)$. In the plots the delayed measurement rises later and slower than the actual motor: in a step test the time constant found from the tick-count velocity is longer by roughly τf, in the Bode plot its phase drops below -90 degrees, and in a closed loop the delay in the feedback reduces the stability margin and increases the overshoot. The tick-timing velocity is delayed by only a few milliseconds (about half of its averaging window plus the 5 ms smoothing), so it follows the motor almost without delay.

The HexaMotor uses a low resolution encoder: 12 lines on the motor shaft, 48 counts per motor revolution in quadrature (4x), and with the 20.4:1 gearbox 979.62 counts per revolution of the output shaft (0.0064 rad per tick). At 15 rad/s the encoder produces about 2340 ticks/s, only 2.3 ticks per 1 ms sample, and at 1 rad/s only one tick every 6.4 ms. Therefore, we use the tick-timing velocity for the measurements and the feedback: it gives a precise velocity with minimum delay, while the tick-count velocity needs the 0.05 s filter.

1.5. Step response modeling

Step response modeling, commonly referred to as the "bump test," is a straightforward examination grounded in the step response characteristics of a stable system. In this test, a step input is applied to the system, and the subsequent response is observed and recorded. To illustrate this concept, imagine a system defined by a particular transfer function:

1.7 $$G(s) = \frac{K}{\tau s+1}$$

The step response in the shown figure is generated using K=10 rad/V and $\tau = 0.1s$.

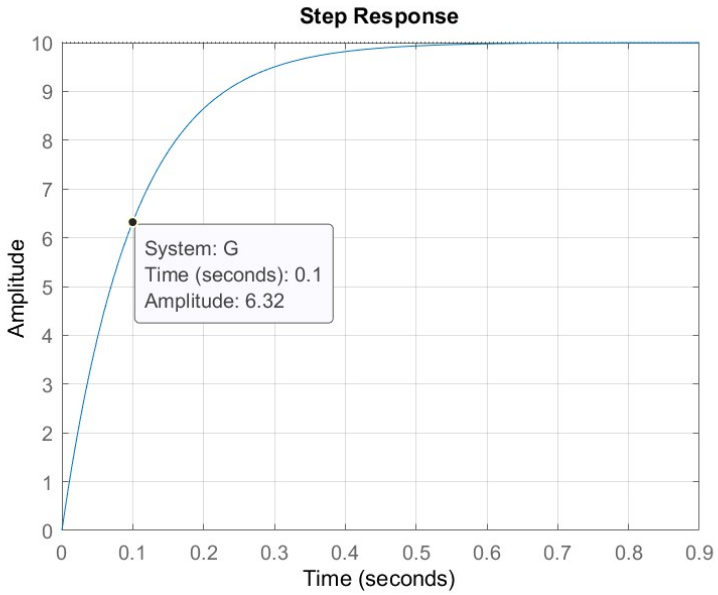


Figure 2 Step response of eq1.7 with K=10, τ=0.05s.

The time constant τ is a measure of how fast the system responds to a change in input. It's defined as the time it takes for the system's response to reach approximately 63.2% of its final value (steady-state value) after a step change in input.

To see why it is the case lets evaluate a first-order linear system that has the general form of a time constant τ and can be described by the following first-order differential equation:

1.8 $$\tau \frac{\mathrm{dy(t)}}{\mathrm{dt}} + \mathrm{y(t)} = Ku(t)$$

If the system is subjected to a step input of magnitude A, the response y(t) can be described by:

1.9 $$y(t) = KA(1 - e^{-\frac{t}{\tau}})$$

By evaluation the response at $t = \tau$ we get:

1.10 $$y(\tau) = KA(1 - e^{-\frac{\tau}{\tau}}) \approx KA \cdot 0.632$$

1.6. Frequency response modeling

System analysis with a sinusoidal input provides insights into the motor's frequency response, resonance behavior, and the effect of damping. The motor's transfer function can be used to investigate how different frequencies of the sinusoidal input are amplified or attenuated, revealing information about the stability, bandwidth, and transient response of the system.

When a sine wave is used as an input for a DC motor, the motor's output results in a sinusoid of the same frequency but is scaled and delayed as by the frequency response theorem. As illustrated in Figure 3, the length of one full period of the sinusoid is represented by t1, while t2 indicates the time delay between the input voltage signal Vm, and the corresponding scaled output speed signal, ω_m.

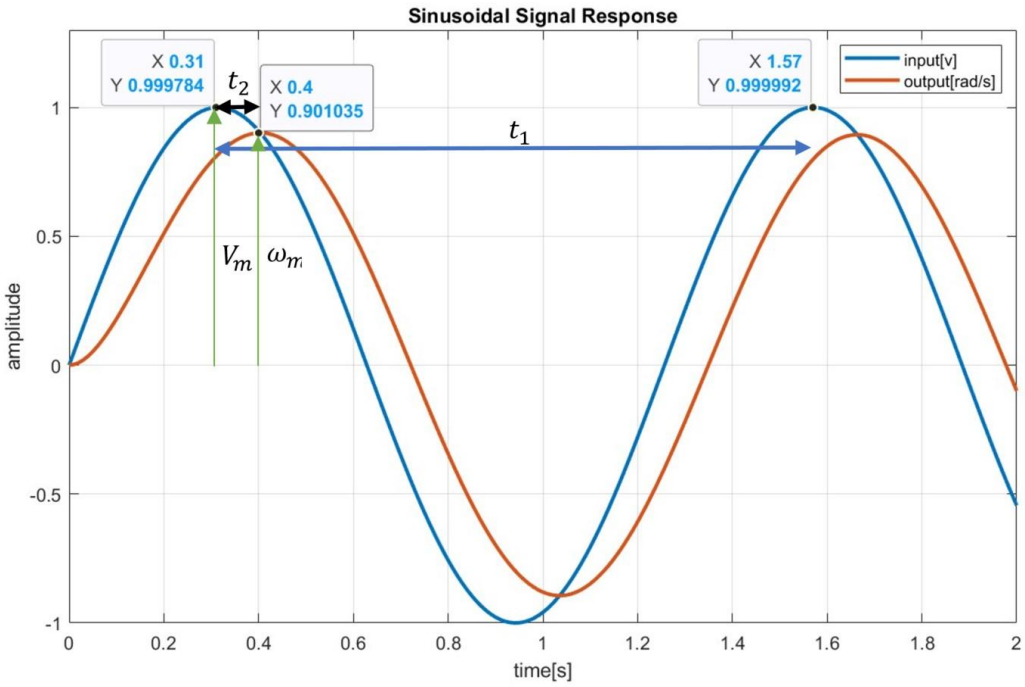


Figure 3 System response with a sinusoidal signal

The magnitude response of the resulting output changes with respect to the frequency of the applied sinusoid and can be used to find the time constant τ of the DC motor with the first order approximation. In addition, a bode plot of the system`s magnitude response can be obtained.

1.11 $$G(s) = \frac{\Omega_m(s)}{V_m(s)} = \frac{K}{\tau s+1}$$

Substituting s = jω the magnitude of the frequency response at a frequency ω of the motor input voltage is then defined as

1.12 $$|G(s)| = \left|\frac{\Omega_m(j\omega)}{V_m(j\omega)}\right| = \left|\frac{K}{\tau j\omega+1}\right| = \frac{K}{\sqrt{\tau^2\omega^2+1}}$$

The system's steady state (or low frequency) gain can then be obtained by setting $\omega = 0$, applying a constant signal:

1.13 $K = |G(0)|$

The phase shift of the system φ_d can be calculated as:

1.14 $\varphi_d = \varphi_{num} - \varphi_{den}$

When φ_{num} and φ_{den} represent the phase angle / delay of the numerator and denominator of the transfer function respectively.

In addition, it can be observed directly from the input output graphs for the sin signal. The phase shift can be expressed as:

1.15 $\varphi_d = -\frac{t_2}{t_1} \cdot 360^{\circ}$

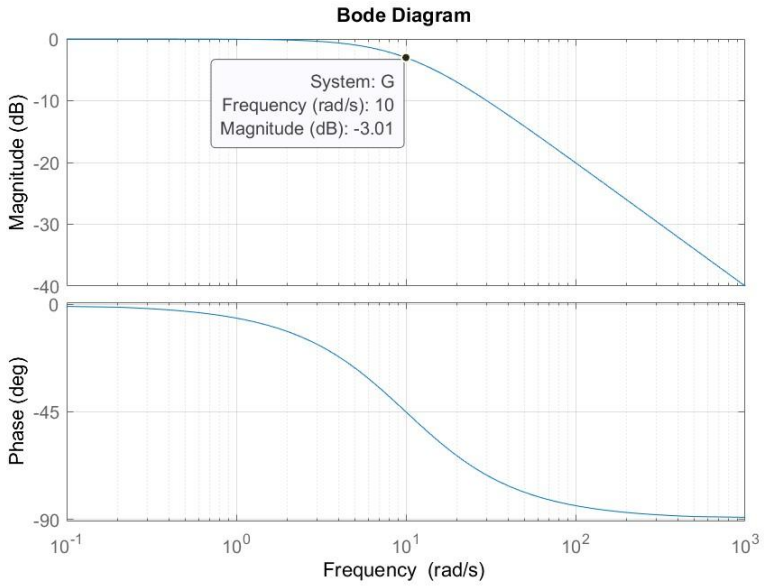


Figure 4 Bode plot of a first order system with $\tau = 0.1$

The Bode plot includes a vital feature known as the cutoff frequency, denoted by ω_b. It's the frequency where the system's gain is 3 dB below its maximum steady-state gain. This 3 dB drop corresponds to the system's half-power point of the system, and defined as $10 \log_{10} 0.5$ leading to a value of approximately -3.01 dB, or equivalently a magnitude of approximately $1/\sqrt{2} \approx 0.70721 \approx 0.707$, or -3.01 dB in terms of the maximum system gain. The cutoff frequency is synonymous with the system's bandwidth and serves as an indicator of the speed at which the system reacts to an input.

1.7. Transient response analysis

In control systems, when a system transitions from one state to another, due to changes in its inputs, it exhibits a certain behavior before settling to its new steady-state. This behavior is termed the transient response. Essentially, it describes how the system reacts to sudden changes, and is a critical aspect to consider, especially when designing systems where rapid and accurate responses are crucial.

Important Parameters in Transient Response:

The performance of control systems during the transient state is often described using the following terms:

- Rise Time (Tr): Time taken for the response to rise from 10% to 90% (or 0% to 100% in some cases) of its final value.
- Peak Time (Tp): Time taken for the response to reach the first peak.
- Overshoot (OS): The amount by which the response exceeds its final steady-state value. It's often expressed as a percentage of the final value.
- Settling Time (Ts): Time taken for the response to stay within a certain percentage (commonly 2% or 5%) of its final value.
- Steady-State Error (Ess): The difference between the desired output and the actual output after the system has settled.

The following figure shows the step response of the system $G(s) = \frac{0.9}{s^2+1.4s+1}$, and marks all the relevant parameters.

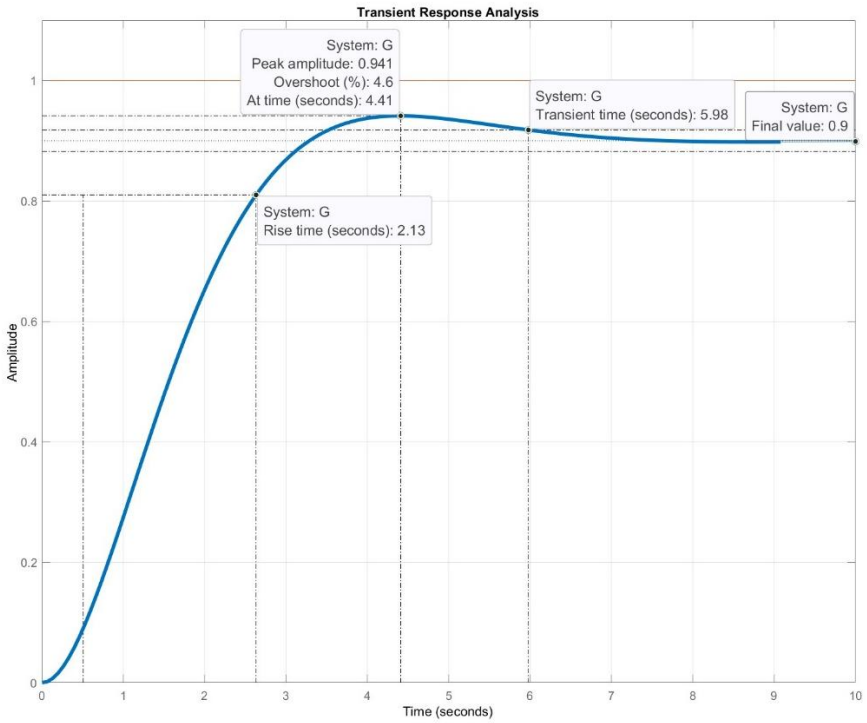


Figure 5 step response of a second order system

1.8. PID controller

The PID controller, which stands for Proportional-Integral-Derivative, is one of the most widely used feedback control algorithms in industry. Its popularity stems from its effectiveness in a broad range of applications and its relatively straightforward tuning process.

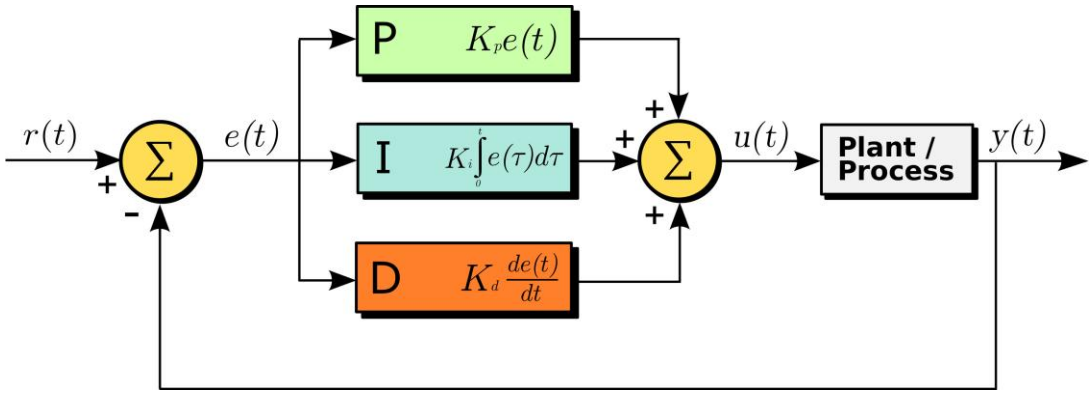


Figure 6 Block diagram of a PID controller

1.16 $$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{\mathrm{d}e(t)}{\mathrm{d}t}$$

The PID controller consists of three main components:

Proportional (P): This part produces a control action that's directly proportional to the error between the setpoint and the measured process variable. The proportional gain Kp determines how aggressively the controller responds to an error.

Integral (I): The integral term is concerned with the accumulated error over time. If there is a persistent, steady-state error, the integral action will build up and eliminate it.

Derivative (D): The derivative action provides a control effort to counteract the rate of error change. It anticipates the future error by its rate of change and provides control action to prevent it from occurring.

For additional reading consider the following links:

Introduction: PID Controller Design
https://ctms.engin.umich.edu/CTMS/index.php?example=Introduction&section=ControlPID

1.8.1. Proportional speed control

In proportional control, the control action is directly proportional to the error between the desired set point and the actual measured value, in this case, the motor speed. The equation for the control action is given by: $u(t) = K_p e(t) = K_p(r(t) - y(t))$,

K_p is the proportional gain. $r(t) = \omega_d(t)$ is the desired angular velocity, $y(t) = \omega_m(t)$ is the measured angular velocity and $u(t) = v_m(t)$ is the control impute (the voltage applied to the motor).

The close loop transfer function can be derived

1.17 $$T(s) = \frac{Y(s)}{R(s)} = \frac{\mathrm{P(s)\cdot C(s)}}{1+\mathrm{P(s)\cdot C(s)}} = \frac{K\cdot K_p}{\tau s+1+K\cdot K_p}$$

Whereas P(s) is the first order approximation of the dc motor. And $\mathrm{C(s)} = K_p$ represent the controller transfer function.

1.8.1. Final value theorem

The Final Value Theorem states that if a system is stable and if the limit exists, the steady-state value of a time-domain function $f(t)$ can be found directly from its Laplace Transform $F(s)$. The theorem is given by:

1.18 $$\lim_{t\to\infty} f\,(t) = \lim_{s\to 0} s\cdot F(s)$$

1.8.2. Steady state error

Steady-state error is the difference between the desired and actual velocity of the DC motor once the transients have died down, and the system has reached its steady-state condition. The main reason for this steady-state error in proportional control lies in the nature of the control law itself. Since the control action is proportional to the error, when the error reduces to zero, so does the control input. In physical systems like a DC motor, there may be disturbances, such as friction, that require continuous control effort to overcome. If the control input becomes zero, these disturbances may cause the system to drift away from the desired set point.

Given a step input $R(s) = \frac{1}{s}$ The error transfer function can be derived as:

1.19 $$E(s) = R(s) - Y(s) = \; R(s) - \mathrm{T(s)\cdot R(s)} = \frac{1}{s} - \frac{K\cdot K_p}{s(\tau s+1+K\cdot K_p)}$$

By applying the final value theorem:

1.20 $$\lim_{t\to\infty} e\,(t) = \lim_{s\to 0} s\cdot E(s) = \lim_{s\to 0} s\left(\frac{1}{s} - \frac{K\cdot K_p}{s(\tau s+1+K\cdot K_p)}\right) = 1 - \frac{K\cdot K_p}{1+K\cdot K_p}$$

The steady state error is given by:

1.21 $$e_{ss} = 1 - \frac{K\cdot K_p}{1+K\cdot K_p}$$

2. Pre Lab

2.1. Step response modeling

a. Given the first order approximation of a dc motor, what would be the values of K and τ (give parametric solution).
b. Using Matlab, plot the step response of the following first order system $G(s) = \frac{1}{0.1s+1}$ Mark the time constant τ and the steady state gain K.

2.2. Frequency response modeling

a. Using Matlab plot the response of the system $G(s) = \frac{1}{0.1s+1}$ to various sin inputs (at list 5 different frequencies), calculate the gain change as a function of frequency from the sin responses. You can use the "lsim" function in Matlab to simulate a system response to an arbitrary input signal in the time domain. Or implement it in Simulink.
b. Approximate the cutoff frequency ω_b and plot the system`s sinusoidal response at this frequency.
c. Using the first order approximation $G(s) = \frac{K}{\tau s+1}$ and by evaluating the magnitude of the transfer function at the cutoff frequency ω_b , derive an expression to determine τ based on ω_b
Hint: $|G(\omega_b)| = \frac{1}{\sqrt{2}}|G(0)|$
d. Using the first order approximation $G(s) = \frac{K}{\tau s+1}$ and by evaluating the phase of the transfer function, derive an expression to determine τ in terms of the frequency input of the signal and the resulting phase delay.
Hint: $\varphi_d = Phase(\frac{K}{\tau s+1})\Big|_{s=j\omega}$

2.3. PID controller

a. Calculate the parametric steady state error for the given system $G(s) = \frac{1}{0.1s+1}$ in a close loop using a proportional gain controller Kp and a step response input.
b. Plot the response of the close loop system using Kp=10, compare the steady state value to the theoretical.
c. Calculate the transfer function of a closed loop velocity controlled dc motor with a PI Controller using the first order approximation for the dc motor transfer function.
d. Calculate the steady state error for the above.
e. For the given system: $G(s) = \frac{1}{0.1s+1}$ Design a PI controller (using any method) with rise time $0.2s > T_r > 0.1s$, overshoot of up to 15% and settling time of up to 0.5s.
f. Consider the DC motor equations, what will happen to the time constant τ in case an additional rotating mass will be added to the motor. Hint: consider what will happen to the inertia **J.**

3. In Lab

3.1. Introduction

Welcome to the lab! Before we dive into the experiments, it's crucial to familiarize yourself with the hardware we'll be working with. The setup you will be working with is called HexaMotor. It includes a DC motor with an encoder connected to an Arduino Uno R4 Minima board and a motor driver. This can be controlled using MATLAB Simulink. Additional information is available in the appendix of this manual.

- Start by opening MATLAB R2026b (with the Arduino support packages and the R4 Minima patch installed, see the Appendix)
- Navigate to …\Documents\MATLAB
- Create a new folder with your name and copy the lab materials into it
- Open "HexaMotor_MotorTest_V03.slx" Simulink model.
- Prior to running the model make sure the setup is connected to the PC through USB cable and the power supply is connected to the socket.
- Make sure the rotating mass is present.
- Under hardware you will find the Monitor&Tune button, press it, if everything is properly set it will compile the Simulink model and upload a C++ code to the micro controller. When the process has finished the button with be switch to a stop button and at the button bar a simulation time will start counting.

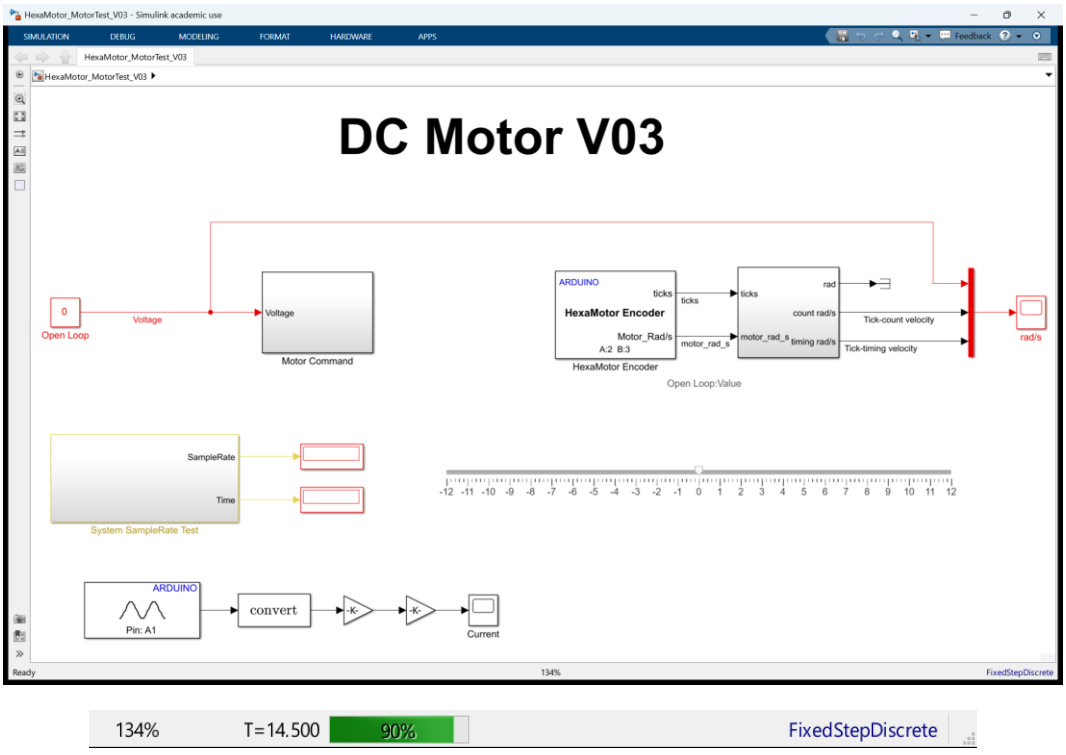


Figure 7 Simulink model for interaction with the HexaMotor Setup

The Simulink model acts as an interface, allowing the interaction with the hardware and monitoring it`s signals.

- Double click on the scope, it will open the signals from the system: the applied voltage to the motor, the Tick-count velocity and the Tick-timing velocity (HexaMotor Encoder block); see Velocity measurement with an encoder in the Introduction.
- Change the open loop voltage, either by writing the open loop block values between [-12 ..12] or using the slide bar.

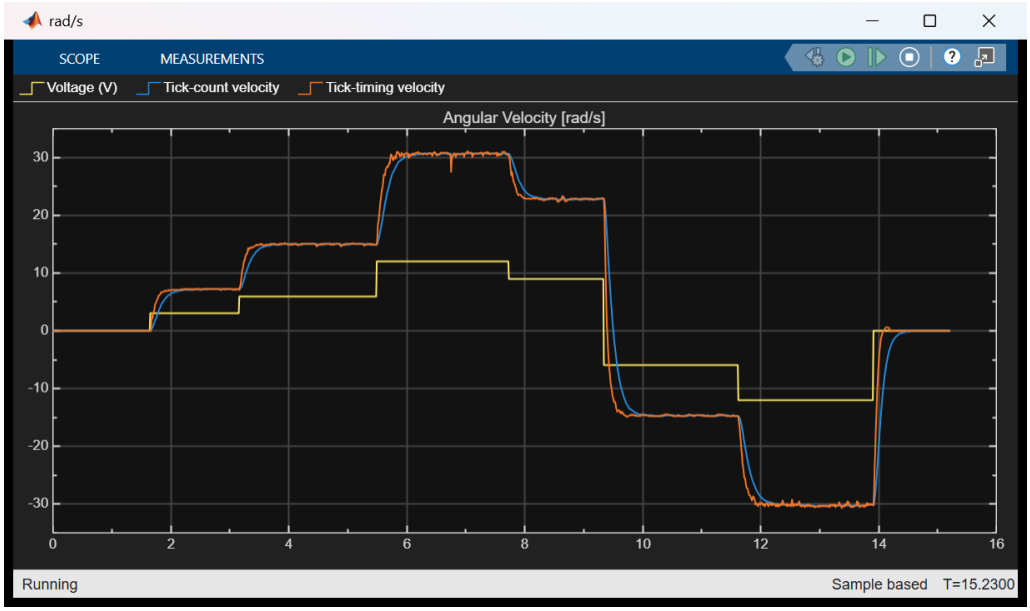


Figure 8 Live scope data from motor response

- On the button left you will notice a block named System SampleRate, the block outputs the simulation sample time and the measured time, and in additional performs I/O toggling which can be check with external hardware to verify the actual Sample Rate of the system.
- And at the top you near the Stop / Monitor & Tune button there is Stop Time, which can be change from inf to any number and will stop the simulation.
- You can export the scope to a figure by pressing File→Print To Figure, in the figure it is possible to make measurements by selecting Tools→Data Tips and clicking the plot. If you press Shift while selecting a data point it will add an additional data point. Then the data points can be moved around. Explore the figure and scope options and save a plot for the report. A good plot contains Title, Lables, Legend, units.

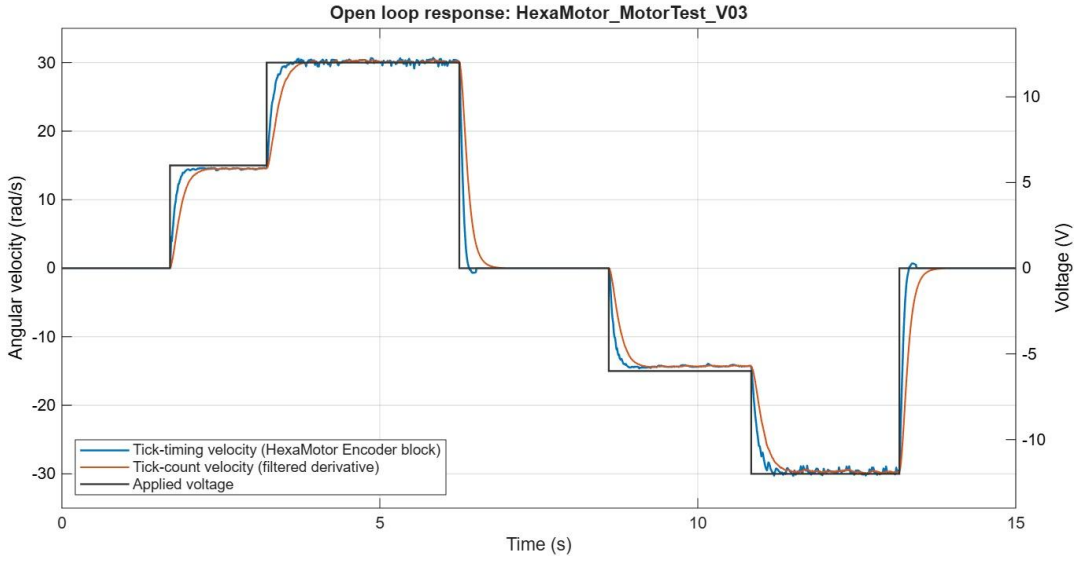


Figure 9 Figure generated from the recorded scope data (ScopeData)

- In addition, the scope data is exported to the Matlab environment under the structure variable ScopeData when the simulation is stopped. You can access the data using the following commands:
  ```
  time = ScopeData.time;
  u = ScopeData.signals.values(:,1);   % input (applied voltage)
  output = ScopeData.signals.values(:,3);   % tick-timing velocity (column 2: tick-count velocity)
  ```
  Or plot it directly using:
  ```
  plot(ScopeData.time,ScopeData.signals.values)
  ```
- In addition, it is possible to save the data for later processing by writing:
  ```
  save record_name ScopeData
  ```
  And retrieving it back to Matlab either by double clicking the saved file or writing:
  ```
  load('record_name.mat')
  ```

Note that you may notice some oscillation at constant voltage, that is due to non-linearity of the friction. If you measure the phenomena you may notice that its frequency is the same as the rotation angle frequency.

For the Report:

a. Present a plot using a figure from the scope simulation
b. Include a plot generated in the Matlab environment, using the ScopeData variable. Don't forget to include legend, Title, Lables and Grid to the plot. You can check how to add this by typing "help plot" in the Matlab environment.
c. Write the max velocity of the motor (at 12V, -12V)

3.2. Coulomb friction

We will begin our analysis by identifying the Coulomb friction within the system. Coulomb friction, often referred to as 'dry friction,' is a constant magnitude friction that opposes motion and is independent

of the relative velocity between surfaces. In the context of a brushed DC motor, this phenomenon manifests as the Dead-Zone. The Dead-Zone is a region in which the motor doesn't produce any movement even though a voltage is applied. This is primarily because the applied voltage or torque is not sufficient to overcome the inherent static friction of the system. It's crucial to identify and compensate for this Dead-Zone to ensure smooth operation and accurate control of the motor, particularly during its startup and low-speed operations

- Using the same "HexaMotor_MotorTest_V03.slx" Simulink model. Find the minimum voltage values required to start the motor motion. You can change the slider range if you wish to use it to find the minimal values.
  You can push the motor a bit with your hand and try to find the least resistance angle. (due to the non-linearity in the friction mentioned earlier).
- Write those values for next section.

3.3. Step response modeling

In this section we will use the bump test to estimate the system`s transfer function. We will use in two methods, one with initial voltage 0, and the other with initial voltage of 2 volts.

- Open the "HexaMotor_System_Identification_V03.slx" Simulink model. Make sure the switch is set to step input.
- Set the Motor Friction Preamp constants: a positive value in the upper constant and the same negative value in the lower constant. Use at most 0.5 V, even if the start voltage you measured in the previous section is higher: the pre-amp is also applied when the command is zero and it switches sign with the command, so a higher value can make the motor jump or oscillate around zero speed. Talk to the TA before setting a value higher than 0.5 V.
- Adjust the step values to step at 1 second, from initial 0-volt to 7-volt step, update the Stop time to 5 seconds and record the response.
- Run the experiment again this time with a step from initial 2-volt to 9-volt step. Record the response.

System Identification

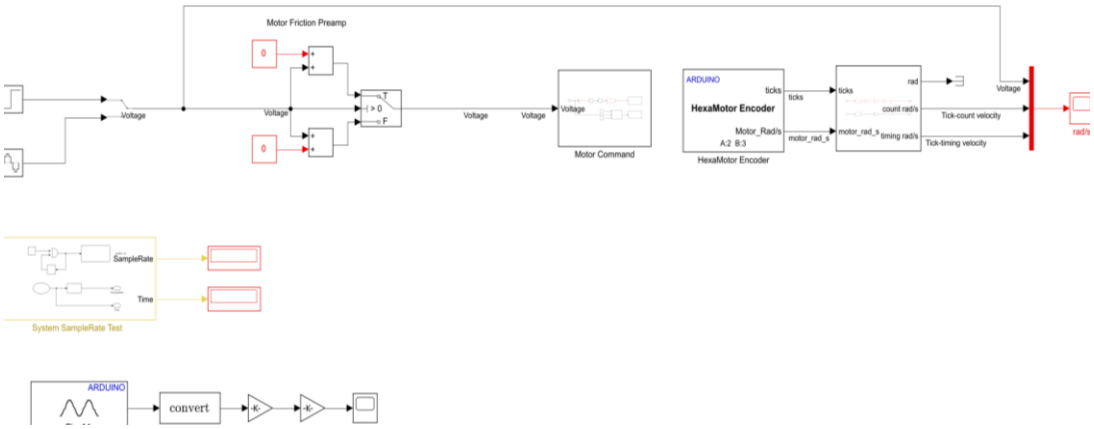


Figure 10 Simulink model : HexaMotor_System_Identification_V03.slx"

For the Report:

a. Find the transfer function from both experiments recording using the first order approximation. If the difference is more than 10% check your calculations. Remember that the step size is 7 volts. And in the second experiment the initial angular velocity isn`t zero. Consider in which cases it is preferred to perform one or the other.

b. The scope also records the tick-count velocity (ScopeData.signals.values(:,2)). Estimate the time constant τ from this signal as well and explain the difference between the two values (see Velocity measurement with an encoder).

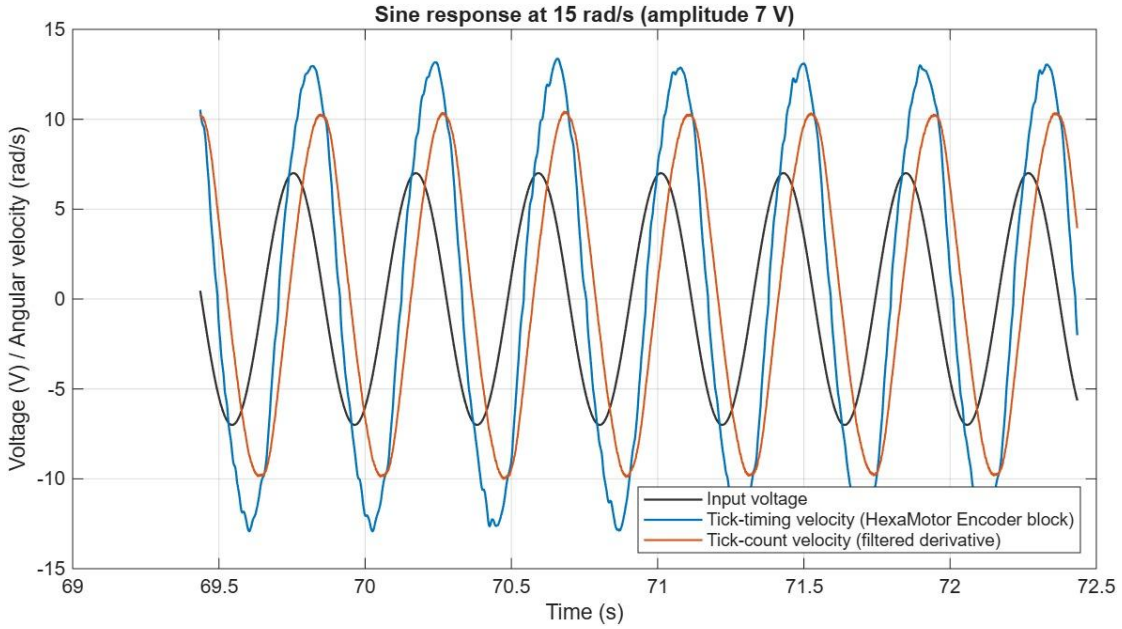
c. Create a plot in Matlab which contains a step response from 0 volts to 7 volts from the recordings and as a simulation using the step function in Matlab for the identified transfer function.
Hint: Use the following commands to simulate a step response similar to the experiment. Don't forget to include title. Legend etc. to the plot.

```
% T Is the calculated transfer function
plot(ScopeData.time,ScopeData.signals.values)
hold on
plot(ScopeData.time+1,step(T,ScopeData.time)*7)
```

3.4. Frequency response modeling

In this section we will generate a bode plot for the system and derive the transfer function using its frequency response to several input frequencies.

- Using the same "HexaMotor_System_Identification_V03.slx" Simulink model. Move the switch to a sin input. And update the stop time to inf.
- Update the parameters of the input Amplitude:7 Frequency (rad/sec): 1 rad/sec and run the model, notice the system response.
- Measure the time delay and magnitude of the response.



For the Report:

a. Fill up the following table make sure to record at list 7 frequencies at the range of [0..30] rad/sec Hint: you can change the frequencies while the model is running and print to figure while it runs. Gain $= \frac{Max\ velocity}{Input\ Amplitude}$, $\omega = 0$ can be calculated from the step response.

Frequency (Rad/sec)	Input Amplitude (V)	Max velocity (rad/s)	Time Delay (s)	Gain (rad/s/v)	Gain(dB).	Phase (degree)
0						
…..						
30						

Figure 11 Sine wave response

b. Approximate the ω_b by evaluating the frequency at which the amplitude $|G(\omega_b)| = \frac{1}{\sqrt{2}}|G(0)|$. store the response for that frequency and calculate the time constant τ. How close it is to the τ calculated from the step response? Which method will give a more precise result?
Hint: Pre lab question 2.2 c.

c. Plot a bode plot using the data from the table.
Hint: use the following Matlab commands:

```
% Create a frequency response data object
% units: amplitudes [rad/s/v], frequencies [Rad/sec], phases [Deg] (negative)
freq_data = idfrd(amplitudes.*exp(1j*deg2rad(phases)), frequencies, 0);
% Estimate transfer function (first order system)
sys = tfest(freq_data, 1)

% plot bode like plot, with the added points
s=tf('s');
w = logspace(-1,2,100);

% For the tick-count velocity (column 2) add the low pass filter of the measurement:
% [mag,phase] = bode(sys*(1/(0.05*s+1)),w);
[mag,phase] = bode(sys,w);   % tick-timing velocity (column 3)

figure(1);
subplot(2,1,1)
semilogx(w,20*log10(squeeze(mag)))
hold on
semilogx(frequencies, 20*log10(amplitudes), '*');

subplot(2,1,2)
semilogx(w,squeeze(phase))
hold on
semilogx(frequencies,phases)
```

Example Plot

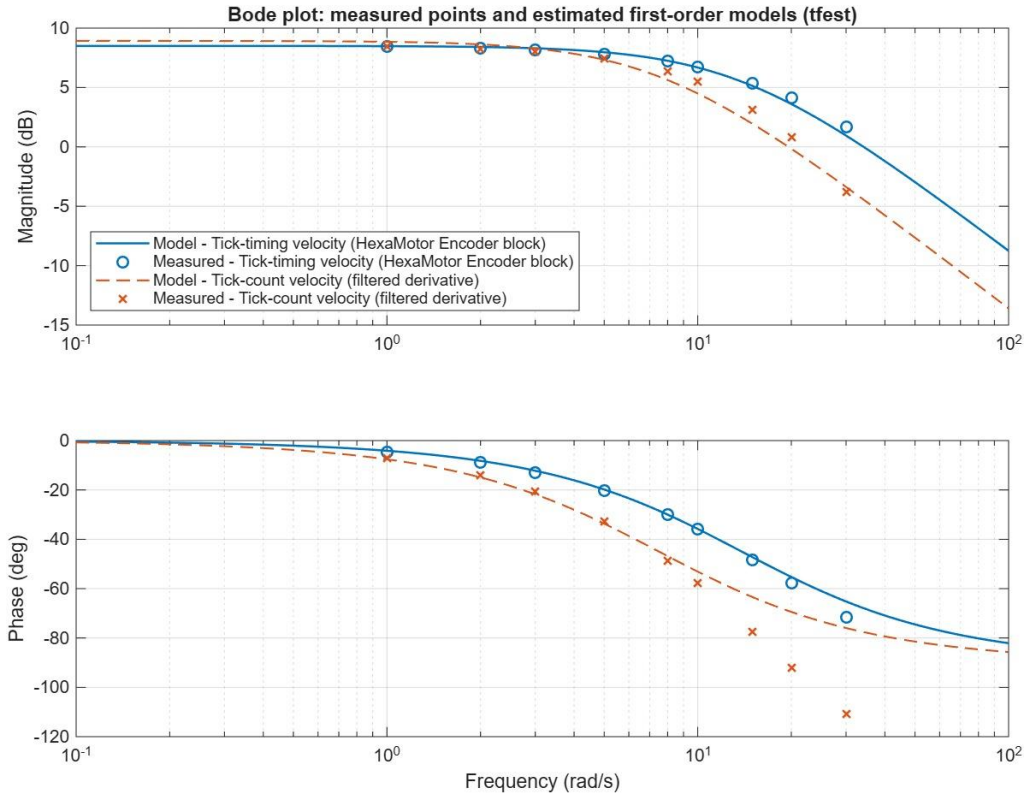


Figure 12 Bode plot containing simulated and measured data

Note: With the tick-timing velocity (column 3) the measured phase stays above -90 degrees, as expected for a first order system. The tick-count velocity (column 2) contains an additional low pass filter at τ=0.05 s, and its phase drops below -90 degrees at higher frequencies. Compare the two measurements in your Bode plot.

d. How close the transfer function derived from the bode estimation to the one found from the step response. What do you think, which one is more precise? which was faster to achieve?

3.5. PID controller

In velocity control setups for motors, a PI (Proportional-Integral) controller is commonly used. While the PID controller includes all three terms — Proportional, Integral, and Derivative — the PI controller only uses the Proportional and Integral terms. The reason for excluding the Derivative term in velocity control is due to the nature of the application. In many motor control scenarios, the Derivative term can introduce noise or lead to excessive sensitivity to rapid changes. Given the objectives of velocity control, the PI controller strikes a balance between performance and stability, delivering smooth and consistent motor speed without the potential drawbacks of the Derivative term.

3.5.1. Steady state error

We will start by measuring the steady-state error of the angular velocity using a simple proportional gain of 1 and comparing the results to a simulated system based on the model identified in previous sections.

- Open the "HexaMotor_Velocity_Control_V03.slx" Simulink model.
- Set the Motor Friction Preamp constants as in the step response section (at most ±0.5 V; talk to the TA before setting a higher value).
- Adjust the square wave Pulse Generator a period of 4 seconds pulse width of 2 seconds and delay of 1 second, set the amplitude to 15 rad/s and update the Stop time of the simulation to 7 seconds.
- Update the PI controller to proportional gain Kp=1, integral gain Ki=0 and derivative gain Kd=0. Leave the Manual Switch on the Tick-timing velocity signal as the feedback.
- Run the experiment.

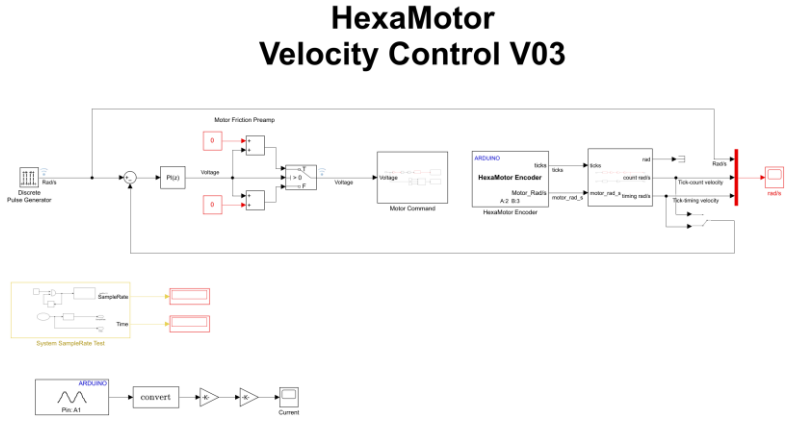


Figure 13 Simulink model : HexaMotor_Velocity_Control_V03.slx

For the Report:

a. Compare the steady state error to the theoretical one
b. Using the recording of the experiments create a plot in Matlab containing the response from the experiment and simulated results from the plant found in section 3.4. You may notice that the responses aren`t exactly the same, that is due to non-linearity's in the actual setup. If you switch the feedback to the Tick-count velocity signal with the Manual Switch, its low pass filter becomes

part of the loop; you may also add simulated results including the low pass filter for comparison ($LPF(s) = \frac{1}{0.05s+1}$).

Hint: use the following Matlab commands: (do not forget to update the transfer function and add grid, axis labels, title to the plot).

```
% Generate plots from recordings and simulated transfer function
s=tf('s');
G = 2.5/(0.1*s+1); % update the estimated transfer function
LPF = 1/(0.05*s+1);
figure(1)
plot(ScopeData.time,ScopeData.signals.values(:,1))
hold on
plot(ScopeData.time,ScopeData.signals.values(:,3),'.');
[y,t] = lsim(G/(1+G),ScopeData.signals.values(:,1),ScopeData.time);
plot(t,y);
[y,t] = lsim(G*LPF/(1+G*LPF),ScopeData.signals.values(:,1),ScopeData.time);
plot(t,y);
```

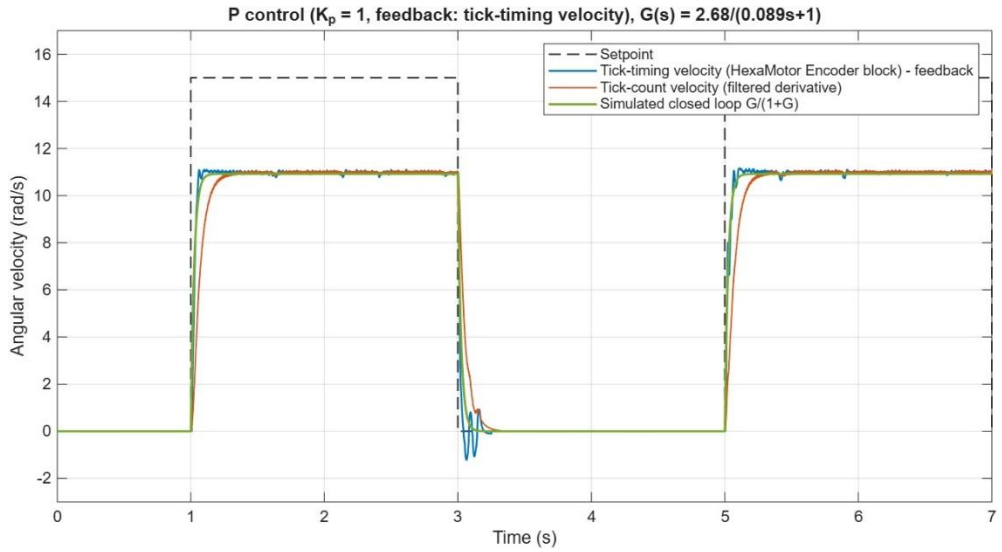


Figure 14 Square wave response, actual system and simulated models

3.5.2. PI Controller manual tuning

In this section, we will discuss how to tune the parameters of a PI (Proportional-Integral) controller to achieve a desired response.

- Begin by setting the integral gain to 0 and adjust the proportional gain within the range of [0 to 1] until the response exhibits a slight overshoot or reaches the desired rise time.
- Introduce the integral gain to minimize the steady-state error and ensure the response meets the desired peak time, overshoot, and settling requirements. If needed, decrease the proportional gain.
- Fine-tune both the Proportional and Integral gains as required to fulfill the design specifications.

Adjust the parameter to get the following control specifications:

$0.3s > \mathrm{T}_{PeakTime} > 0.15s$, $\mathrm{T}_{Settling}(5\%) < 0.5s$, $Overshoot < 15\%$

and no sustained oscillation around zero speed while the set point is zero.

Hint: you can set the Stop time of the simulation to 'inf' while tunning the parameters.

Note: At 15 rad/s the measured velocity has a small ripple of about ±2% caused by the friction in the gearbox, therefore use the 5% band for the settling time. A Motor Friction Preamp above 0.5 V, or a very high proportional gain, can make the motor oscillate around zero speed while the set point is zero (a limit cycle): every time the controller output changes sign, the pre-amp switches between its positive and negative constants. If you observe it, check that the pre-amp is at most 0.5 V and reduce the proportional gain, and explain its origin in the report.

For the Report:

a. Plot the response of the system, with the simulated theoretical model. Mark the important criteria and calculate overshoot of the system.

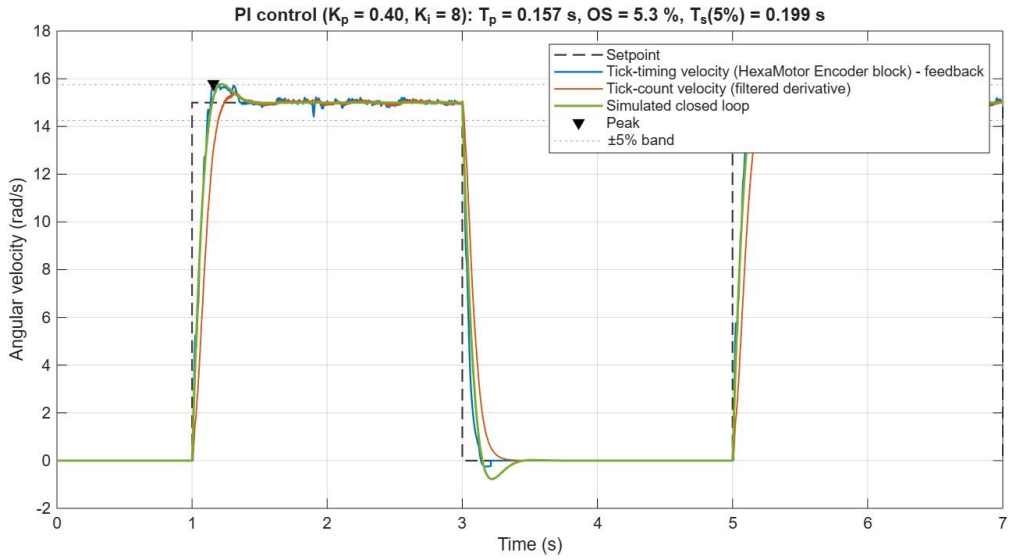


Figure 15 Square wave response, actual system and simulated models

Note: The measured response closely follows the simulated first-order model. The feedback is the tick-timing velocity, measured directly from the time between encoder ticks: single deviating ticks are rejected and the result is lightly smoothed, so almost no delay is added to the feedback. The tick-count velocity, shown for comparison, lags behind because of its 0.05 s filter. The remaining differences come from the friction and the friction pre-amp, which the linear model does not include (for example around the stop at 3 s). When the tick-count velocity is selected as the feedback with the Manual Switch, the filter pole (τ=0.05 s) is close to the motor pole and a second-order model is needed to match the response.

4. Appendix

Github Page: https://github.com/TALs-Education/HexaMotor

4.1. DC Motor

This gearmotor consists of a low-power, 12 V brushed DC motor combined with a 20.4:1 metal spur gearbox, and it has an integrated 48 CPR quadrature encoder on the motor shaft, which provides 979.62 counts per revolution of the gearbox's output shaft.

Product page: https://www.pololu.com/product/4883

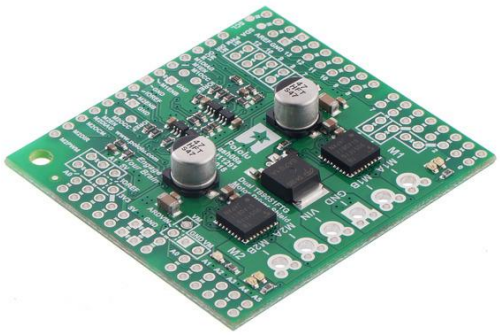
Data Sheet: https://www.pololu.com/file/0J1829/pololu-25d-metal-gearmotors.pdf

4.2. Arduino board / Motor Driver

The Arduino Uno R4 Minima board (Renesas RA4M1, 48 MHz Arm Cortex-M4) reads the encoder and drives the motor through the Pololu Dual TB9051FTG Motor Driver Shield for Arduino. The motor is connected to channel M2 (PWM on pin 10, direction on pin 8), and the motor current is measured on A1 through the current sense output of the shield (500 mV/A). The encoder channels are connected to pins 2 and 3.

Uno R4 Minima: https://docs.arduino.cc/hardware/uno-r4-minima

Dual TB9051FTG Motor Driver Shield for Arduino: https://www.pololu.com/product/2520



Dual TB9051FTG Motor Driver Shield for Arduino (image: Pololu)

4.3. Simulink Support package

With MATLAB and Simulink Support Packages for Arduino® hardware, you can use MATLAB and Simulink to interactively communicate with your Arduino. Simulink also enables you to perform model deployment for standalone operations on Arduino boards.

MathWorks: https://www.mathworks.com/hardware-support/arduino.html

4.4. R4 Minima patch

The Arduino Uno R4 Minima needs a patch of the Arduino support packages (rotary encoder, PWM and analog reference support for the Renesas RA4M1). Run applyPatch.m from the R4MinimaPatch folder once, with MATLAB started as administrator, and restart MATLAB. The patch also adds the HexaMotor Encoder block to the MATLAB path. See R4MinimaPatch/README.md for the details.

4.5. HexaMotor Encoder block (tick-timing velocity)

The HexaMotor Encoder block measures the angular velocity from the time between encoder pulses. Every encoder edge is time stamped in an interrupt, and the velocity is the inverse of the moving average of the last N edge intervals (N = 48, one motor revolution, limited to the last 10 ms), with single deviating edges rejected and a light 5 ms smoothing. Unlike the tick-count velocity it does not need a strong low pass filter, so the measured response is faster. The block also outputs the encoder count, which is used for the angle output.

4.6. 3D Model – Fusion360

The setup is design in Fusion360 as an open source project, for educational purposes. Available at:

Fusion360: https://a360.co/3WoIGKZ